



Dipartimento di INGEGNERIA INFORMATICA,
MODELLISTICA, ELETTRONICA E SISTEMISTICA

Corso di Laurea in Ingegneria Informatica

AIOps e Model Context Protocol

Bridge architetturale tra LLM e hypervisor

Relatore:

Prof. Sergio Greco

Correlatore:

Lucio La Cava

Candidato:

Michele Arnoni

Matricola: 220121

Anno Accademico 2025/2026

Indice

1	Introduzione	1
1.1	Contesto e scenario generale	1
1.1.1	Hypervisor	1
1.1.2	Model Context Protocol (MCP)	1
1.2	Problematiche che si propone di risolvere	1
1.3	Obiettivi della tesi	2
2	Background	3
2.1	Evoluzione delle ITOps e AIOps	3
2.2	Infrastructure as Code (IaC) e API REST	4
2.3	Interfacce per LLM	5
2.4	Limiti degli approcci attuali	6
2.5	Rassegna delle tecnologie alternative	6
3	Analisi dei Requisiti	8
3.1	Attori e Casi d'Uso	8
3.1.1	Attori	8
3.1.2	Casi d'Uso Principali	8
3.2	Requisiti Funzionali	11
3.2.1	Interfacciamento con Hypervisor e Protocollo MCP	11
3.2.2	Ispezione e Monitoraggio dell'Infrastruttura (Read-Only)	12
3.2.3	Gestione del Ciclo di Vita delle Istanze (Operazioni di Stato)	12
3.2.4	Sicurezza e Controllo d'Esecuzione	13
3.3	Requisiti Non Funzionali	13
4	Progettazione e scelte architetturali	15
4.1	Architettura	15
4.1.1	Visione statica del sistema: Component Diagram	15
4.1.2	Visione dinamica del sistema: Sequence Diagram	17
4.1.3	Modello di Comunicazione	17

4.2	Scelte progettuali	18
5	Sviluppo e implementazione	20
5.1	Implementazione dei Casi d'Uso (UC)	20
5.2	Soddisfazione dei Requisiti	21
5.2.1	Requisiti Funzionali	21
5.2.2	Requisiti Non Funzionali	22
5.3	Dettagli operativi	23
5.4	Sfide tecniche	24
6	Testing e valutazione sperimentale	25
6.1	Soddisfazione dei criteri di test	25
6.2	Metriche di performance e benchmark	25
6.3	Analisi dei risultati	25
7	Conclusioni e sviluppi futuri	26
7.1	Conclusioni oggettive	26
7.2	Sviluppi futuri	26
	Bibliografia	27

Sommario

L'integrazione dei *Large Language Model (LLM)* può offrire nuove opportunità per la semplificazione nella gestione e nella manutenzione di infrastrutture server IT.

In particolare, tramite l'emergente protocollo *MCP Server (Model Context Protocol)*, è possibile fornire ad un *LLM* un layer di comunicazione standardizzato con un sistema IT. Questo approccio abilita interazioni basate sul linguaggio umano con tutta una serie di vantaggi in termini di semplicità e immediatezza operativa per task che tradizionalmente richiederebbero approcci più impegnativi e complessi.

Tuttavia, è importante considerare la natura aleatoria e non deterministica intrinseca di un sistema basato su AI, pertanto questo lavoro di tesi illustra lo sviluppo di un *server MCP* che permette il relazionarsi di un *LLM* con un *Hypervisor Prox-mox*, con annessa valutazione dei limiti e benefici che tale architettura comporta, ponendo una rigorosa attenzione alla mitigazione dei rischi e alla sicurezza. Attraverso la validazione di questo Proof of Concept, il lavoro dimostra come l'impiego del protocollo *MCP*, unito ad adeguati vincoli architetturali, permetta di governare l'esecuzione di task potenzialmente distruttive in modo controllato e affidabile.

Capitolo 1

Introduzione

1.1 Contesto e scenario generale

1.1.1 Hypervisor

In ambito infrastrutturale IT, una macchina di tipo server opera grazie all'impiego di sistemi operativi specialistici, noti come *Hypervisor* [1], specializzati nella virtualizzazione multipla di ambienti disaccoppiati tra loro e su cui gireranno i vari servizi.

1.1.2 Model Context Protocol (MCP)

Un qualsiasi *LLM* può interagire con software classici interpretandoli come tools e instaurando con essi una comunicazione simil API, ma molto più standardizzata. In questo scenario l'azienda *Anthropic* ha sviluppato un protocollo chiamato *MCP* [2], che permette interazioni strutturate basate sulla logica di un sistema client-server, rispettivamente *LLM* e il layer *MCP*.

Attualmente il progetto *MCP* è opensource ed è stato donato da *Anthropic*, per garantirne l'indipendenza, alla *Agentic AI Foundation* [3] [4], un fondo gestito dalla *Linux Foundation*.

1.2 Problematiche che si propone di risolvere

La gestione di un *hypervisor* richiede competenze sistemistiche verticali e l'uso di interfacce GUI o CLI spesso complesse e poco user-friendly. In questo lavoro di tesi si propone di realizzare e valutare un *MCP server* con il ruolo di bridge architetturale tra un *LLM* e un hypervisor aggiungendo così una nuova interfaccia per l'utente basata sul linguaggio naturale.

1.3 Obiettivi della tesi

L'architettura non si limita ad interfacciare un *LLM* ad un hypervisor, ma prevede soluzioni volte a irrobustire questa interfaccia con scelte di design che gestiscono operazioni potenzialmente distruttive e mantengono il controllo sulle azioni dell'*LLM*.

Capitolo 2

Background

2.1 Evoluzione delle ITOps e AIOps

Tutte le operazioni volte alla gestione delle infrastrutture IT, quali manutenzione, ottimizzazione e pianificazione delle operazioni strategiche a lungo termine, sono descritte dal termine *ITOps* (*IT Operations*) [5], termine ormai consolidato nel settore, ma che ha subito profonde trasformazioni con l'avvento del cloud computing e la crescente complessità dei sistemi distribuiti.

Tradizionalmente, le attività di ITOps venivano gestite in modo prevalentemente manuale. Gli operatori facevano affidamento su cruscotti di monitoraggio statici e sistemi di alert automatizzati basati su parametri preimpostati.

Con l'esplosione dei dati generati dalle infrastrutture moderne (log, metriche, tracce) e la transizione verso architetture a microservizi, questo approccio si è rivelato difficilmente scalabile e con un'inefficienza direttamente proporzionale alle dimensioni del sistema.

La frammentazione dei servizi e l'enorme volume di alert possono rendere molto rumorosa e lenta l'identificazione della causa radice (*root cause analysis*) di un disservizio.

Per rispondere a queste sfide, negli ultimi anni si è assistito all'introduzione delle *AIOps* (*Artificial Intelligence for IT Operations*) [6]. Questo termine, coniato da *Gartner* [7], una delle più importanti e influenti società di ricerca e analisi nel campo IT, descrive come l'impiego di software di monitoring o interazione basato su AI all'interno dei flussi di lavoro delle ITOps possa portare a benefici concreti in termini di incremento dell'immediatezza operativa e di gestione della grossa mole di alert.

L'obiettivo principale delle AIOps è aumentare le performance di identificazione e risoluzione di disservizi in tempi più rapidi e di incapsulare automazioni complesse

ed eterogenee in comandi impartiti tramite il linguaggio naturale.

Le soluzioni AIOps operano tipicamente attraverso diverse fasi:

- **Raccolta e aggregazione:** centralizzazione di grandi volumi di dati eterogenei provenienti da molteplici sorgenti dell'infrastruttura.
- **Analisi e rilevamento:** utilizzo di algoritmi di machine learning per l'interpretazione dei dati in tempo reale.
- **Correlazione:** raggruppamento di eventi correlati per ridurre il rumore di fondo ed evitare la duplicazione delle segnalazioni, demoltiplicando gli alert.
- **Risoluzione automatica:** attivazione immediata di workflow automatizzati per mitigare o risolvere il problema senza l'intervento umano diretto o invio di proposte di risoluzione automatizzate con scelta a cura di un umano per le problematiche più critiche.

L'evoluzione verso le AIOps rappresenta quindi un passo fondamentale per garantire la resilienza e l'efficienza dei sistemi IT moderni, ponendo le basi per una gestione infrastrutturale semplificata e strutturata.

2.2 Infrastructure as Code (IaC) e API REST

L'*IaC* (*Infrastructure as Code*) [8] è una pratica ingegneristica che prevede di trattare una infrastruttura IT come un software, con tutti i vantaggi che ne conseguono.

Il workflow di un team IT basato sul paradigma *IaC* si compone delle seguenti fasi:

- **Writing:** scrittura dei file di configurazione architetturale tramite linguaggi tipo *HCL*, *YAML* o *JSON*.
- **Versioning:** i rilasci del codice sorgente vengono strutturati tramite sistemi di controllo delle versioni come *Git*, incorporando anche test di controllo automatici ad ogni nuova versione.
- **Provisioning:** un engine di automazione legge i file di configurazione e ne applica le specifiche in maniera idempotente, ovvero senza duplicare azioni già applicate in precedenza o riassegnare ripetutamente le stesse risorse.
- **Distributing:** si applicano script per la distribuzione dell'infrastruttura in vari ambienti assicurando coerenza e stabilità grazie all'uso dei file sorgente già redatti e testati nella fase di Writing.

È facile intuire come questo approccio, più strutturato e di impronta fortemente ingegneristica, porti concreti vantaggi in termini di controllo del sistema, robustezza, manutenibilità, resilienza all'errore umano, riproducibilità delle configurazioni e scalabilità del sistema.

Ponendo particolare attenzione alla fase di Provisioning e alle tecnologie che la popolano, è possibile imbattersi in strumenti di automazione specifici al sistema su cui operano, come *AWS CloudFormation*, *Azure Resource Manager* o *Google Cloud Deployment Manager*, oppure in strumenti universali come *Terraform* [9]. Questi strumenti si interfacciano con il sistema tramite *API (Application Programming Interface)* permettendo interazioni dirette e programmate ed evitando le interazioni manuali tramite console.

Questo lavoro di tesi vuole evolvere questi strumenti grazie all'AI, rendendoli più flessibili e potenti, integrando un *MCP* come layer di comunicazione verso *API REST* permettendo ad un LLM di dialogare con i sistemi IT.

2.3 Interfacce per LLM

I Large Language Model sono di base incapaci di interagire con software esterni, per superare questi limiti architetturali ogni provider ha sviluppato in autonomia un proprio paradigma di interazione tra cui spicca il modello *ReAct (Reasoning and Acting)* introdotto da Yao et al. [10]. Questo pattern istruisce il modello a produrre un output che alterna in modo iterativo tracce di ragionamento logico (Reasoning) ad azioni specifiche (Acting) rivolte a sistemi esterni, permettendo all'agente di aggiornare dinamicamente il proprio contesto.

L'implementazione tecnica del pattern ReAct si concretizza tramite il meccanismo del *Tool Calling* (o *Function Calling*), un'idea che prevede di iniettare la lista strutturata in formato JSON dei tools a disposizione dell'IA con tanto di descrizione di ogni funzione chiamabile insieme ai suoi parametri.

La pipeline di funzionamento si compone quindi nei seguenti step:

- **Decisione:** l'agente determina che per soddisfare una richiesta è necessario interagire con un tool
- **Invocazione:** l'LLM genera un *payload* JSON contenente l'intenzione di invocare un *tool* specifico e i relativi argomenti tipizzati.
- **Esecuzione:** l'applicazione chiamante intercetta, gestisce la richiesta e reinietta il risultato nel contesto dell'LLM.

- **Sintesi** l’LLM riprende con la sua generazione per terminare la sintetizzazione della risposta

2.4 Limiti degli approcci attuali

Sebbene il *Tool Calling* offra un’interfaccia tra i modelli generativi e i sistemi IT, la sua implementazione non filtrata in ambienti di produzione critici solleva sfide ingegneristiche non trascurabili. Dal momento in cui un sistema dipendente da un hypervisor *bare-metal* come Proxmox VE è spesso impiegato in ambito enterprise l’esecuzione di comandi errati può arrecare danni economici importanti.

La principale causa di comandi distruttivi deriva dal fenomeno delle ”allucinazioni” che può colpire un qualsiasi LLM o da un prompt ambiguo non deterministicamente eseguibile.

Un *Large Language Model*, a causa della sua natura probabilistica, potrebbe generare *payload* JSON formalmente validi ma semanticamente dannosi.

Negli approcci tradizionali risulta complesso implementare un rigoroso *Principio del Minimo Privilegio*, poiché la logica di validazione e filtraggio delle intenzioni pericolose ricade interamente sul client applicativo.

In secondo luogo, l’implementazione *custom* dei *tool* crea un forte accoppiamento (*tight coupling*) tra la logica decisionale dell’agente e le API dell’infrastruttura con il rischio di trovarsi con un’architettura poco scalabile e molto rigida.

Infine, le integrazioni convenzionali mancano di un meccanismo per l’assegnazione dinamica delle funzionalità in quanto solitamente l’elenco degli strumenti e delle risorse disponibili deve essere inserito staticamente nel prompt di sistema (*hardcoded*). Questo impedisce all’infrastruttura di esporre o revocare dinamicamente le capacità operative in base allo stato corrente del server o ai privilegi dell’utente richiedente. Questa rigidità strutturale e l’assenza di un perimetro di sicurezza delimitato evidenziano la necessità di un layer di astrazione intermedio tra l’intelligenza artificiale e le API di sistema.

2.5 Rassegna delle tecnologie alternative

Per ovviare ai limiti del pattern ReAct e del tool calling in un contesto di produzione critica, esistono diverse tecnologie e approcci alternativi che potrebbero essere considerati. Alcune di queste includono:

- **API Gateways:** un gateway API può fungere da intermediario tra l’LLM e le API di sistema, garantendo un livello di sicurezza aggiuntivo e consentendo

il controllo di accesso e l'autenticazione. Gli API gateways supportano anche l'aggregazione di dati da più servizi, il routing di chiamate e la gestione di errori, riducendo la complessità di sviluppo e l'accoppiamento tra l'agente e le API.

- **Orchestratori e Automazioni:** strumenti come Ansible, Puppet, e SaltStack, possono essere utilizzati per orchestrazione di flussi di lavoro e automazione di task. Questi strumenti possono essere utilizzati per creare pipeline di lavoro predefinite e sicure per l'invocazione di specifici comandi sul sistema.
- **Microservizi e FaaS (Function as a Service):** implementare una serie di microservizi o funzioni specifiche che gestiscono le operazioni critiche può rendere il sistema più modulare e sicuro. Questo approccio permette di separare l'aspetto decisionale del modello di linguaggio dalle operazioni concrete sul sistema, rendendo più facile implementare il principio del minimo privilegio e un accesso condizionale. È però importante considerare che il demando della traduzione delle intenzioni dell'IA e la validazione dei payload JSON interamente al codice *custom* comporta un notevole onere di sviluppo e manutenzione.
- **Middleware di sicurezza:** esistono soluzioni specifiche di middleware come Keycloak o OAuth2 Proxy che possono essere utilizzate per aggiungere una layer di sicurezza aggiuntivo tra l'LLM e le API di sistema. Questi strumenti possono gestire l'accesso e l'autenticazione di sicurezza, riducendo il rischio di allucinazioni o comportamenti ambigui.
- **Sistemi a Plugin Proprietari:** i principali provider di LLM di frontiera hanno tentato di risolvere il problema incorporando protocolli proprietari (come gli ormai deprecati OpenAI Plugins), che permettevano al modello di interrogare API esterne tramite la lettura di file di specifica (come OpenAPI). Tali soluzioni, seppur funzionali, generano un forte *vendor lock-in* e risultano inadatte ad architetture on-premise chiuse come i cluster Proxmox.

Capitolo 3

Analisi dei Requisiti

3.1 Attori e Casi d'Uso

A scopo di contestualizzazione del progetto è necessario presentare i principali Attori che interagiscono con il Server MCP e i principali scenari d'uso:

3.1.1 Attori

- **Hypervisor bare-metal:** Ambiente di virtualizzazione installato direttamente sull'hardware con cui si ha necessità di interagire.
- **Client LLM:** Applicazione client su cui è eseguita l'interfaccia del Large Language Model incaricato di tradurre le richieste dell'utente in linguaggio naturale a payload JSON-RPC.
- **Utente (Amministratore di Sistema):** Utente che deve eseguire transazioni di gestione e manutenzione sull'hypervisor tramite il linguaggio naturale.

3.1.2 Casi d'Uso Principali

I casi d'uso descritti sono affrontati con granularità SEA-LEVEL.

- **UC1 - Ispezione e monitoraggio dell'infrastruttura:** L'Utente richiede una panoramica sullo stato operativo o sulle risorse del cluster di virtualizzazione attraverso il Client LLM.
 - **Attori Coinvolti:** Utente (iniziatore), Client LLM (mediatore), Hypervisor (sistema secondario).

- **Precondizioni:** Il Server MCP è attivo, in ascolto sul canale di trasporto designato e autenticato con successo verso l'API REST dell'Hypervisor tramite token segregato.
- **Flusso Principale degli Eventi:**
 1. L'Utente formula una richiesta sullo stato di una o più risorse.
 2. L'LLM interpreta la richiesta, seleziona i tool necessari esposti dall'MCP e genera un corrispondente payload JSON-RPC per il Server MCP.
 3. Il Server MCP intercetta e valida la richiesta in JSON-RPC (*RF2*).
 - * Se il payload JSON-RPC risulta sintatticamente corretto il sistema continua con la gestione della richiesta.
 - * Se il payload JSON-RPC risulta sintatticamente errato il sistema restituisce un messaggio di errore JSON-RPC all'LLM.
 4. Il Server MCP invoca gli endpoint REST dell'Hypervisor necessari a soddisfare la richiesta.
 5. L'Hypervisor elabora la richiesta ed emette la risposta con codice HTTP 200 contenente le metriche grezze.
 - * Se l'hypervisor risulta irraggiungibile viene restituito un errore di Connection Timeout.
 6. Il Server MCP estrae e normalizza i dati salienti, restituendoli al Client LLM sotto forma di *Observation* strutturata (*RF3*, *RF4*).
 7. Il Client LLM sintetizza i dati ricevuti e genera la risposta in linguaggio naturale per l'Utente.
- **Postcondizioni:** Nessuna modifica di stato viene apportata all'infrastruttura; l'operazione di lettura e le relative chiamate API vengono registrate nel log di sistema (*RF5*).
- **UC2 - Gestione del ciclo di vita e provisioning delle istanze (VM o Container LXC):** L'Utente desidera la creazione di una istanza di VM o Container LXC specifica.
 - **Attori Coinvolti:** Utente (iniziatore), Client LLM (mediatore), Hypervisor (sistema secondario).
 - **Precondizioni:** Il Server MCP è attivo, in ascolto sul canale di trasporto designato e autenticato con successo verso l'API REST dell'Hypervisor tramite token segregato.

– **Flusso Principale degli Eventi:**

1. L'Utente formula la richiesta di voler creare una istanza di VM o Container LXC all'LLM, comunicando nella richiesta anche quale risorse si vogliono allocare.
2. L'LLM interpreta la richiesta, seleziona i tool necessari esposti dall'MCP e genera un corrispondente payload JSON-RPC per il Server MCP.
 - * Nel caso non siano state definite tutte le risorse necessarie, l'LLM provvede a richiederle all'Utente prima di compilare un JSON-RPC.
3. Il Server MCP intercetta e valida la richiesta in JSON-RPC (*RF2*).
 - * Se il payload JSON-RPC risulta sintatticamente corretto il sistema continua con la gestione della richiesta.
 - * Se il payload JSON-RPC risulta sintatticamente errato il sistema restituisce un messaggio di errore JSON-RPC all'LLM.
4. Il Server MCP invoca gli endpoint REST dell'Hypervisor necessari a soddisfare la richiesta.
5. L'Hypervisor elabora la richiesta ed emette la risposta con codice HTTP 200 contenente l'esito della richiesta.
 - * Se l'hypervisor risulta irraggiungibile viene restituito un errore di Connection Timeout.
6. Il Server MCP estrae e normalizza l'esito dell'operazione, restituendolo al Client LLM sotto forma di *Observation* strutturata (*RF3*, *RF4*).
7. Il Client LLM sintetizza i dati ricevuti e genera la risposta in linguaggio naturale per l'Utente.

– **Postcondizioni:** Le altre istanze di virtualizzazione non vengono alterate, le relative chiamate API vengono registrate nel log di sistema (*RF5*).

- **UC3 - Esecuzione controllata di operazioni critiche (*Human-in-the-Loop*):** L'Utente desidera l'eliminazione di una certa istanza di virtualizzazione (Figura 4.3).

– **Attori Coinvolti:** Utente (iniziatore), Client LLM (mediatore), Hypervisor (sistema secondario).

- **Precondizioni:** Il Server MCP è attivo, in ascolto sul canale di trasporto designato e autenticato con successo verso l'API REST dell'Hypervisor tramite token segregato.
- **Flusso Principale degli Eventi:**
 1. L'Utente formula la richiesta di voler eliminare una certa istanza di virtualizzazione.
 2. L'LLM interpreta la richiesta, seleziona i tool necessari esposti dall'MCP e genera un corrispondente payload JSON-RPC per il Server MCP.
 3. Il Server MCP intercetta e valida la richiesta in JSON-RPC (*RF2*), ma riconosce in essa la presenza di comandi critici per soddisfarla (*RF11*).
 4. Il Server sospende l'esecuzione dell'API verso l'Hypervisor e restituisce all'LLM una richiesta formale di validazione, impedendo l'azione silente (*RF6*).
 5. L'LLM informa l'Utente che è necessaria una conferma esplicita e restituisce un riepilogo dell'istanza che verrà eliminata, al fronte di prevedere errori umani.
 6. L'Utente esamina il riepilogo ed eventualmente conferma la richiesta.
 7. L'LLM inoltra la conferma al Server MCP in attesa.
 8. Il Server MCP una volta validata la conferma procede con l'inoltro dei comandi verso l'Hypervisor.
 9. L'Hypervisor effettua l'eliminazione e restituisce l'esito al Server MCP.
 10. Il Server MCP comunica l'effettiva eliminazione all'LLM che a sua volta notifica l'utente.
- **Postcondizioni:** L'istanza interessata viene eliminata, le istanze di virtualizzazione non interessate non vengono alterate, le relative chiamate API inclusa la conferma vengono registrate nel log di sistema (*RF5*).

3.2 Requisiti Funzionali

3.2.1 Interfacciamento con Hypervisor e Protocollo MCP

- **RF1 - Esposizione dinamica dei tool (*Tool Discovery*):** Il layer MCP deve esporre in modo dinamico i tool per le interazioni con l'infrastruttura IT in formato *JSON Schema*.

- **RF2 - Layer di traduzione (JSON-RPC verso REST):** Il layer MCP si occupa di tradurre dei payload JSON-RPC generati dall'LLM e controllare se conformi ai suoi standard per poi inoltrare le eventuali richieste REST API all'hypervisor.
- **RF3 - Formattazione dei risultati per il contesto del LLM client:** Il server MCP dovrà restituire un output formattato in modo strutturato e fruibile per il client LLM.

3.2.2 Ispezione e Monitoraggio dell'Infrastruttura (Read-Only)

- **RF4 - Monitoraggio dello stato hardware e logico del sistema:** Il sistema deve offrire la possibilità di ricevere una panoramica completa del sistema in termini di risorse allocate ed occupate in tempo reale dai nodi nonchè dalle VM e Container in servizio.
- **RF5 - Tracciamento delle transazioni (*Audit Logging*):** Il layer MCP deve mantenere uno storico di tutte le richieste ricevute in termini di utente, data e ora, JSON-RPC, API invocate per soddisfare la richiesta e codice HTTP ricevuto.
- **RF6 - Sistema di Notifica:** All'atto di una richiesta che necessita intervento umano, il sistema deve inviare un alert all'utente interessato.

3.2.3 Gestione del Ciclo di Vita delle Istanze (Operazioni di Stato)

- **RF7 - Controllo dell'alimentazione (*Power Management*):** Devono essere esposti strumenti di controllo dell'alimentazione di VM e Container in modo sicuro (*boot, shutdown, reboot*).
- **RF8 - Provisioning di base:** Devono essere esposti strumenti per la creazione, gestione dinamica delle risorse allocate e distruzione controllata di VM e Container, anche tramite template registrati sull'hypervisor.
- **RF9 - Gestione degli snapshot:** Prima di ogni intervento di manutenzione deve essere consigliata dall'LLM la possibilità di eseguire uno snapshot del sistema, consentendo un eventuale ripristino *rollback* all'occorrenza.

3.2.4 Sicurezza e Controllo d'Esecuzione

- **RF10 - Autenticazione presso l'hypervisor e gestione dei permessi:** Il layer MCP deve caricare le credenziali d'accesso dell'utente attraverso variabili d'ambiente per poi mantenere un token di autenticazione valido per interagire con l'hypervisor. Le credenziali non devono transitare tramite il contesto conversazionale dell'LLM.
- **RF11 - Intercettazione e blocco di comandi distruttivi:** Il sistema deve saper intercettare e richiedere conferma all'utente nel caso certi prompt prevedano comandi possibilmente distruttivi o che possano apportare modifiche architetturali importanti.

3.3 Requisiti Non Funzionali

- **RNF1 - Modularità ed Estendibilità:** Deve essere possibile interfacciare diversi LLM con diversi Hypervisor e prevedere un'architettura tale da rendere semplice aggiungere o deprecare funzionalità dell'MCP Server.
- **RNF2 - Efficienza delle prestazioni (latenza):** La piattaforma deve introdurre il minimo *overhead* computazionale e garantire che un certo prompt esegua i comandi sul sistema e riceva un output in un tempo trascurabile.
- **RNF3 - Disponibilità (*Availability*):** La piattaforma deve garantire un alto livello di disponibilità anche sotto elevato stress di carico.
- **RNF4 - Concorrenza:** La piattaforma deve prevedere paradigmi di gestione delle concorrenze e operazioni asincrone non bloccanti.
- **RNF5 - Tolleranza ai guasti (*Fault Tolerance*):** L'architettura deve essere tale da implementare una gestione robusta degli imprevisti, quali possibili timeout di risposta, errori o JSON malformati e incoerenti.
- **RNF6 - Consistenza e Pattern Saga:** Logicamente ove un prompt preveda più comandi, esso deve essere considerato eseguito solo se tutti i comandi hanno successo, ma in termini pratici diventa ingegneristicamente complesso trasformare un prompt in una transazione (*ACID*). Per questo motivo, bisogna quindi che il sistema interrompa eventuali sequenze di comandi nel caso uno dovesse fallire e proporre eventualmente delle *operazioni di rollback*.

- **RNF7 - Sicurezza e supervisione (*Human-in-the-Loop*):** Bisogna che azioni critiche vengano monitorate dall'utente in tempo reale e all'occorrenza validate da esso. Tutte le azioni possibilmente distruttive o che possano apportare modifiche profonde al sistema devono ricevere conferma da un utente umano. Deve inoltre essere previsto, quando possibile, il tempo di down del sistema su alcuni comandi complessi ed esporre la scelta all'utente.
- **RNF8 - Manutenibilità:** Il codice deve essere strutturato affinché in caso di manutenzione il sistema rimanga in down in tempi brevi. Deve essere implementato un sistema di logging granulare sullo storico di ogni prompt per utente e sulle chiamate API che ne sono conseguite in modo da avere il controllo completo e documentato su tutte le azioni dell'LLM.

Capitolo 4

Progettazione e scelte architettureali

4.1 Architettura

4.1.1 Visione statica del sistema: Component Diagram

Progettazione della suddivisione per proprietà logice dei vari moduli del sistema.

- Topologia del sistema:

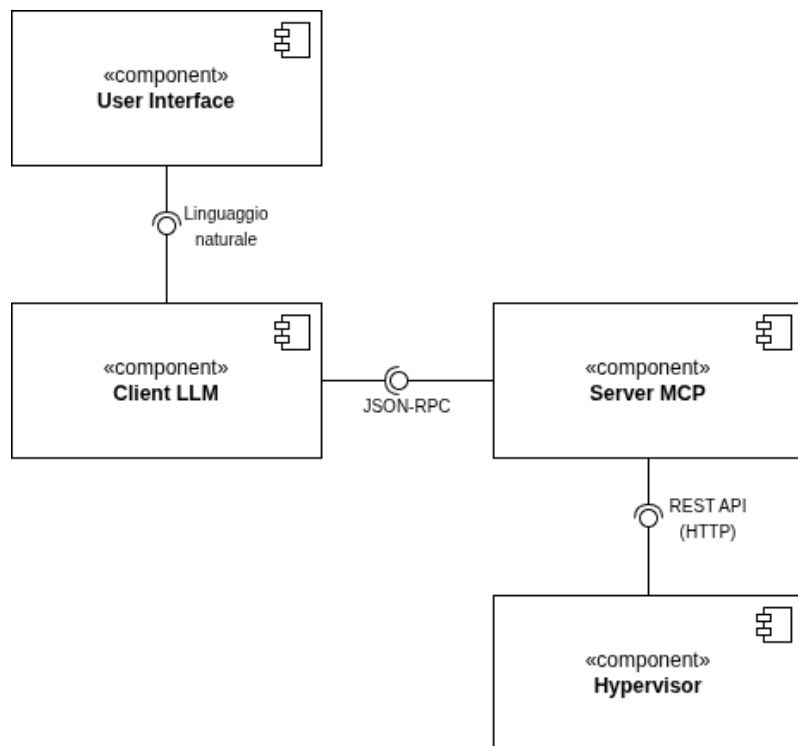


Figura 4.1: Component Diagram dell'intero sistema.

- Architettura interna del Middleware (MCP):

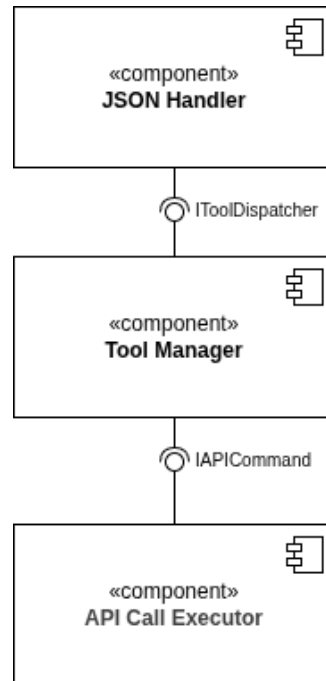


Figura 4.2: Component Diagram del Server MCP.

Come illustrato nel diagramma dei componenti interni (Figura 4.2), il Server MCP è progettato seguendo un approccio modulare per la separazione delle responsabilità di parsing, logica applicativa e comunicazione di rete. Esso si articola in tre componenti logici sequenziali:

- **JSON Handler:** Costituisce il punto di ingresso del middleware. Ha il compito di intercettare il flusso di dati dal Client LLM, validare la conformità del messaggio allo standard JSON-RPC e verificare che i parametri forniti rispettino i *JSON Schema* attesi. Una volta validato, il payload viene decodificato e inoltrato al livello logico successivo attraverso l'interfaccia **IToolDispatcher**.
- **Tool Manager:** Rappresenta il *core* logico del sistema. Riceve la richiesta strutturata e si occupa di validarne il contesto operativo. In questo modulo risiede la logica di controllo degli accessi e l'implementazione del pattern *Human-in-the-Loop*: se il tool richiesto prevede un'azione classificabile come distruttiva, il manager sospende l'esecuzione e genera una richiesta di conferma. Superati i controlli, formula il comando di backend e lo delega tramite l'interfaccia **IAPICommand**.

- **API Call Executor:** È il modulo incaricato dell'interazione con l'esterno. Riceve il comando astratto dal Tool Manager e lo traduce in chiamate HTTP/REST autenticate verso l'Hypervisor. Questo componente gestisce la latenza di rete, cattura le eccezioni o i codici di errore HTTP e si occupa di impacchettare le metriche ricevute in un' *Observation* restituibile a ritroso fino all'LLM.

4.1.2 Visione dinamica del sistema: Sequence Diagram

Di seguito una panoramica del flusso di funzionamento del sistema tramite la rappresentazione del Sequence Diagram del (*UC3 - Esecuzione controllata di operazioni critiche*), in modo da poter prendere visione di una delle casistiche più complesse e cruciali del sistema:

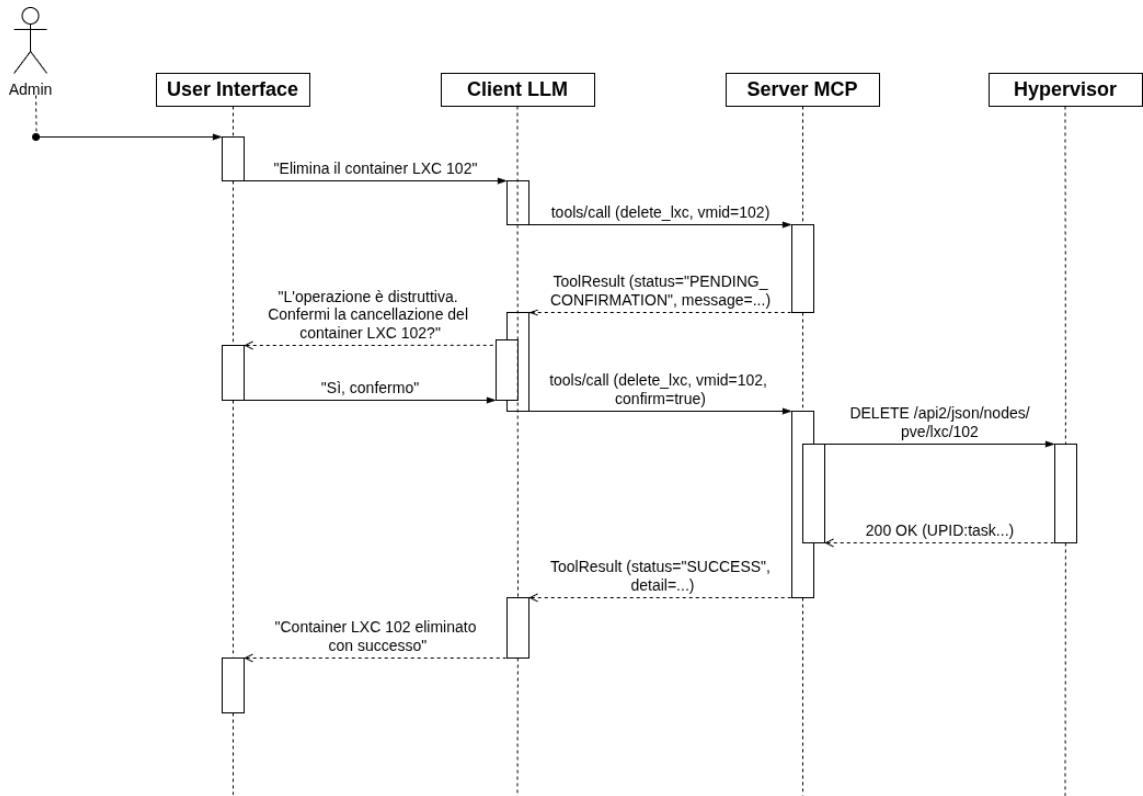


Figura 4.3: Sequence Diagram dell'UC3

4.1.3 Modello di Comunicazione

I dati attraversano la pipeline dei moduli tramite diversi cambi di protocollo affinché ogni modulo possa interpretare le richieste in modo corretto.

- **Utente $\longleftrightarrow$ Client LLM (Linguaggio Naturale):** L'interazione avviene tramite l'interfaccia conversazionale; l'operatore esprime gli intenti operativi o di diagnostica senza dover conoscere la sintassi delle API sottostanti.
- **Client LLM $\longleftrightarrow$ Server MCP (Protocollo JSON-RPC 2.0):** Il modello linguistico converte l'intento dell'utente in invocazioni strutturate di tool (*Tool Calls*) serializzate in payload JSON-RPC conformi alla specifica MCP, scambiate tipicamente tramite canali di standard I/O (`stdio`) o Server-Sent Events (SSE).
- **Server MCP $\longleftrightarrow$ Hypervisor Proxmox VE (REST API su HTTPS):** Il Middleware traduce le chiamate JSON-RPC ricevute in richieste HTTP formalizzate (verbi GET, POST, PUT, DELETE) indirizzate agli endpoint dell'API REST di Proxmox VE, includendo nei relativi header le credenziali di autenticazione (*API Token*).

4.2 Scelte progettuali

A questo punto è necessario introdurre lo stack tecnologico di implementazione e di ambiente:

- **Proxmox VE:** La scelta dell'hypervisor è ricaduta su una soluzione open-source ampiamente utilizzata sia in ambito enterprise che in quello consumer, corredata dall'esposizione di una REST API solida e ben documentata. Quest'ultimo rappresenta un requisito fondamentale affinché il middleware possa orchestrare le risorse.
- **Python 3.11+:** Per quanto riguarda il linguaggio di implementazione del middleware, è stato reputato appropriato l'uso di Python (versione 3.11+) per le qualità che lo contraddistinguono come lo standard *de facto* per lo sviluppo nell'ambito dell'*AI* e dell'automazione di sistema. La sua adozione ha permesso di sfruttare un ecosistema di librerie ormai maturo per il networking (richieste SSH, HTTP), in particolare:
 - *proxmoxer*: Questo wrapper traduce l'albero degli endpoint REST di Proxmox in chiamate a oggetti Python dinamici, evitando la gestione manuale degli header dei pacchetti, dei token di autenticazione e del parsing JSON.
 - *paramiko*: L'uso di questa libreria rappresenta una soluzione architetturale per aggirare i limiti delle REST API esposte, permettendo l'esecuzione

di comandi da riga di comando (*CLI*) tramite il protocollo SSHv2, utile all'instaurazione di tunnel criptati host-to-guest.

- **Anthropic MCP SDK:** Come già discusso nei capitoli precedenti, l'esposizione dei comandi Python all'LLM è delegata al protocollo MCP di Anthropic tramite l'adozione dell'SDK ufficiale. Questo gestisce nativamente il ciclo di vita del server MCP, la negoziazione delle capacità, l'invocazione dei tool e la formattazione in JSON-RPC del flusso di comunicazione.
- **Claude Client:** Le interazioni Utente-LLM avvengono tramite il client ufficiale di Claude per desktop, che permette l'iniezione del server MCP tramite un file locale di configurazione JSON.

Capitolo 5

Sviluppo e implementazione

In questo capitolo vengono analizzate le fasi di sviluppo e implementazione del middleware basato sul Model Context Protocol (MCP). Si descrive come avviene la traduzione dei requisiti precedentemente definiti nelle fasi di design in moduli software concreti e sulle strategie adottate per superare i limiti infrastrutturali emersi durante le fasi di test.

5.1 Implementazione dei Casi d'Uso (UC)

Le funzionalità del Server MCP sono state implementate sulla base dei casi d'uso definiti nel *Capitolo 3*.

- **UC1 - Ispezione e monitoraggio dell'infrastruttura:** Il tool messo a disposizione dell'LLM per la ricezione di una panoramica di tutte le istanze di virtualizzazione è stato chiamato `list_vms()`. Recupera la lista di tutte le Macchine Virtuali (VM) e i container (LXC) presenti sul server Proxmox, inclusi i loro ID, nomi e lo stato di accensione.
- **UC2 - Gestione del ciclo di vita e provisioning delle istanze (VM o Container LXC):** Questo caso d'uso è stato suddiviso in due tool:
 - `manage_vm_state` permette di gestire il ciclo di vita delle istanze tramite i comandi di `start`, `stop` e `reboot`.
 - `clone_resource` permette la creazione di una nuova istanza di virtualizzazione tramite clonazione di una esistente.
- **UC3 - Esecuzione controllata di operazioni critiche (Human-in-the-Loop):** All'interno di questo caso d'uso sono inclusi i tool già descritti `manage_vm_state` e `clone_resource`, ma il più esplicativo è il tool `execute_cmd`, che consente

di eseguire comandi arbitrari CLI direttamente sul nodo tramite SSH, ma ovviamente ha un livello altissimo di criticità in quanto il rischio di allucinazioni può intaccare concretamente il sistema, per questo nelle *docstring* del tool è esplicitamente richiesto all'LLM di chiedere categoricamente conferma per ogni comando all'utente.

5.2 Soddisfazione dei Requisiti

Approcci di implementazione dei requisiti delineati in fase di design nel *Capitolo 3*.

5.2.1 Requisiti Funzionali

- **Interfacciamento e Traduzione (RF1, RF2, RF3):** L'esposizione dinamica dei tool è gestita nativamente dall'SDK MCP, che elabora le funzioni Python in formato JSON Schema. Il layer di traduzione si occupa di processare i payload JSON-RPC generati dall'LLM e di formattare le risposte dell'hypervisor in strutture fruibili per il contesto del modello.
- **Monitoraggio (RF4):** La capacità di ispezione (Read-Only) è soddisfatta dal tool `list_vms`. Questo strumento interroga le API REST di Proxmox restituendo uno snapshot delle risorse attualmente allocate e dello stato di nodi e istanze di virtualizzazione.
- **Tracciamento delle transazioni (RF5):** Il requisito di Audit Logging è soddisfatto dall'infrastruttura del Server MCP che, operando come intermediario, può intercettare e registrare tramite il modulo `logging` di Python ogni transazione JSON-RPC in ingresso e le relative invocazioni REST in uscita, garantendo uno storico documentato delle operazioni.
- **Ciclo di Vita e Provisioning (RF7, RF8):** Le operazioni di alimentazione (boot, shutdown, reboot) e di provisioning sono implementate i tool `manage_vm_state` e `clone_resource`, quest'ultimo in particolare permette la creazione di istanze di virtualizzazione tramite clonazione di istanze esistenti, dal momento in cui una creazione da zero necessiterebbe di troppi parametri complessi e alta probabilità di allucinazioni da parte dell'LLM.
- **Gestione degli Snapshot (RF9):** La raccomandazione preventiva degli snapshot non è delegata a controlli hardware rigidi, ma è implementata a livello semantico tramite *Prompt Engineering*. Le *docstring* dei tool critici istruiscono il modello LLM a suggerire proattivamente all'utente l'esecuzione

di un backup prima di procedere con alterazioni di stato, sfruttando le capacità di ragionamento (ReAct) dell'agente.

- **Sicurezza, Autenticazione e Controllo (RF6, RF10, RF11):** Per soddisfare il requisito di autenticazione (RF10), le credenziali vengono caricate attraverso variabili d'ambiente locali, evitando il loro transito nel contesto conversazionale dell'LLM. Il blocco di comandi distruttivi (RF11) e l'attivazione del sistema di alert verso l'utente (RF6) sono garantiti dall'adozione del pattern Human-in-the-Loop che delega al Client la notifica all'utente.

5.2.2 Requisiti Non Funzionali

- **Modularità ed Estendibilità (RNF1):** Il protocollo MCP disaccoppia logicamente l'LLM dalle API di sistema. Questa astrazione permette di interfacciare il middleware con diversi Client LLM e di aggiungere o deprecare funzionalità indipendentemente dal modello che dovrà interagire con esso.
- **Efficienza (RNF2):** L'implementazione del middleware introduce un overhead computazionale minimo. L'utilizzo del protocollo standardizzato JSON-RPC garantisce che le invocazioni di sistema avvengano con tempi di latenza trascurabili.
- **Concorrenza e Disponibilità (RNF3, RNF4):** La gestione delle operazioni asincrone e non bloccanti è garantita nativamente dal design dell'Anthropic MCP SDK. Sfruttando la libreria `asyncio` di Python, il server è in grado di processare chiamate concorrenti senza bloccare il thread principale, assicurando un alto livello di disponibilità (Availability) anche in scenari di interazione rapida e ripetuta.
- **Tolleranza ai guasti (RNF5):** La gestione degli imprevisti (come timeout di rete o JSON incoerenti) è assicurata dai classici costrutti `try...except`. Mentre le eccezioni e i codici di errore HTTP non interrompono il ciclo di vita del server, anzi vengono incapsulati e restituiti all'LLM come una normale risposta, sotto forma di errore leggibile, delegando all'agente la generazione di una strategia risolutiva da proporre all'utente o eventualmente una semplice argomentazione sull'errore incontrato.
- **Consistenza e Pattern Saga (RNF6):** In assenza di transazioni ACID tradizionali, la consistenza è orchestrata dall'LLM stesso. Qualora un comando in una sequenza dovesse fallire (restituendo un'eccezione dal middleware),

l'LLM interrompe autonomamente la catena di esecuzione, informando l'utente e proponendo azioni di rollback mirate, emulando così il comportamento del *Pattern Saga*.

- **Sicurezza e Supervisione (RNF7):** L'architettura esclude l'esecuzione autonoma di alterazioni di stato critiche. L'infrastruttura forza l'integrazione dell'utente nel flusso operativo (Human-in-the-Loop), garantendo la convalida manuale di ogni operazione potenzialmente distruttiva.
- **Manutenibilità e Logging (RNF8):** L'uso dei decorator Python (es. `@mcp.tool()`) rende il sistema estremamente modulare e manutenibile. L'aggiunta, la modifica o la deprecazione di uno strumento avviene in modo isolato, permettendo di ridurre al minimo i tempi di inattività (downtime) durante gli aggiornamenti del codice.

5.3 Dettagli operativi

In questo paragrafo vengono citati alcuni particolari sul funzionamento dell'intero sistema e vengono colmati possibili dubbi su alcune dinamiche di contorno.

- **File di configurazione:** Per rendere disponibile l'MCP Server è necessario caricare il percorso del file Python che lo definisce come variabile d'ambiente nel file di configurazione del Client, nello specifico per *Claude desktop* è necessario modificare il file *claude_desktop_config.json*. Nello stesso file vengono anche definite le credenziali di accesso a Proxmox tramite token generato dalla console di quest'ultimo, anche se per l'accesso tramite SSH c'è anche bisogno di definire le credenziali effettive.
- **Deferred tools:** Di base l'LLM non inizializza i tool custom finché non gli viene richiesto esplicitamente dall'utente, che innesca l'invocazione del comando `tool_search`.
- **Docstring:** All'interno di un metodo Python è possibile definire dei prompt che verranno letti dall'LLM alla chiamata allo scopo di fornire direttive e contesto, in particolare per questo progetto sono state definite chiare direttive per l'implementazione del paradigma *human in the loop* per i tool possibilmente critici, al fine di non delegare all'LLM la categorizzazione basata sulla criticità, ma sarà lui stesso a leggerlo nelle *docstring* del tool.
- **Struttura dei return:** I metodi Python restituiscono la loro risposta all'LLM come Stringa, ma per migliorarne la qualità è stato definito uno scheletro di

contestualizzazione dei dati, soprattutto per i casi in cui è previsto un codice di errore.

5.4 Sfide tecniche

- **Limiti delle REST API:** Alcune funzionalità non sono esposte dalle REST API di Proxmox, pertanto è necessario utilizzare direttamente comandi shell tramite SSH, ciò è stato possibile tramite la libreria Python *paramiko*, che però non supporta l'uso del token di autenticazione, ma necessita delle credenziali effettive. Inoltre è necessario sottolineare che i comandi lanciati sull'Hypervisor tramite questa tecnica sono eseguiti sul nodo e non sulle singole istanze di virtualizzazione, come invece avviene tramite API.
- **Gestione delle eccezioni:** A differenza del codice tradizionale in questo progetto è necessario gestire le eccezioni con un approccio diverso dal solito, dal momento in cui a prenderne visione e a gestirle non è un utente, ma un LLM, che deve interpretare correttamente e senza ambiguità i possibili messaggi di errore.

Capitolo 6

Testing e valutazione sperimentale

6.1 Soddisfazione dei criteri di test

6.2 Metriche di performance e benchmark

6.3 Analisi dei risultati

Capitolo 7

Conclusioni e sviluppi futuri

7.1 Conclusioni oggettive

7.2 Sviluppi futuri

Bibliografia

- [1] IBM - Stephanie Susnjara e Ian Smalley. *hypervisor overview*. URL: <https://www.ibm.com/it-it/think/topics/hypervisors>.
- [2] Anthropic. *MCP announcement*. Nov. 2024. URL: <https://www.anthropic.com/news/model-context-protocol>.
- [3] Linux Foundation. *MCP Agentic Ai Foundation*. URL: <https://aaif.io/projects/model-context-protocol>.
- [4] Linux Foundation. *MCP Project Page*. URL: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro>.
- [5] IBM. *ITOps overview*. URL: <https://www.ibm.com/it-it/think/topics/it-operations>.
- [6] IBM. *AIOps overview*. URL: <https://www.ibm.com/it-it/think/topics/aiops>.
- [7] Gartner. *AIOps abstract insights*. URL: <https://www.gartner.com/en/documents/5398763>.
- [8] IBM - Jim Holdsworth e Annie Badman. *Infrastructure as Code overview*. URL: <https://www.ibm.com/it-it/think/topics/infrastructure-as-code>.
- [9] IBM/HashiCorp - Derek Robertson - Matthew Kosinski - Gregg Lindemulder. *Terraform topic*. URL: <https://www.ibm.com/it-it/think/topics/terraform>.
- [10] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *The Eleventh International Conference on Learning Representations (ICLR)*. 2023. URL: <https://arxiv.org/abs/2210.03629>.